VECTOR

Security Module

Technical Reference

Version 3.2.0

Authors	visrie, vistbe
Status	Released

Document Information

History

Author	Date	Version	Remarks
visrie	2020-03-11	3.0.0	General rework
visrie	2020-09-21	3.1.0	Added chapter 4.1 Updated attribute list in Table 4-9
vistbe	2020-12-03	3.2.0	Added chapter 4.3.3; Added information about CRC Written Get

Reference Documents

No.	Source	Title	Version
[1]	HIS	API IO Library	2.0.3
[2]	HIS	HIS Security Module Specification	1.1
[3]	Vector	HexView Reference Manual	1.12.05
[4]	RSA	PKCS #1: RSA Cryptography Standard	2.1
[5]	IETF	HMAC: Keyed-Hashing for Message Authentication RFC2104	Feb 1997
[6]	ANSI	Keyed Hash Message Authentication Code X9.71	2000
[7]	Vector	MICROSAR CSM – Technical Reference	4.01.00

Scope of the Document

This technical reference describes the general use of the Security Module software.

Contents

1 Introduction 8

2 Software Architecture 9
2.1 Security Module Components 9

3 Integration 10
3.1 Scope of Delivery 10
3.2 Include Structure 10

4 Configuration 11
4.1 General 11
4.2 Job 11
4.3 Cryptographic Keys 12
4.3.1 Key 12
4.3.2 Key Values 13
4.3.3 Key Operations 14
4.4 CRC 14
4.5 Seed/Key 15
4.5.1 Seed 15
4.5.2 Key 16
4.6 Encryption/decryption 17
4.7 Verification 18
4.7.1 Monolithic Verification 18
4.7.2 Stream Verification 20
4.8 Configuration files 21

5 Preparation of the Application 22
5.1 Key Files 23
5.1.1 Authenticity file 23
5.1.2 INI-Key file mode 25
5.2 HexView 26

6 Functional Description 27
6.1 Key handling 27
6.2 Verification Module 27
6.2.1 Description of the verification process 27
6.3 CRC Module 28
6.3.1 Variants 29
6.3.2 CRCTotal 29
6.3.3 Speed / size optimization 29

6.3.4 Operation mode flags 29
6.4 Encryption / Decryption Module 29
6.4.1 Variants 30
6.4.2 Using an explicit initialization vector 30
6.4.3 Internal / external Key 31
6.4.4 Operation mode flags 31
6.4.5 Operation mode modifiers 32
6.5 Seed / Key 32
6.5.1 Variants 32
6.5.2 Default and extended API 32
6.5.3 Seed generation 32
6.5.4 Key computation 32

7 API Description 33
7.1 Type Definitions 33
7.1.1 General Types 33
7.1.2 Structures used by Verification 34
7.1.3 Structures used by CRC 35
7.1.4 Structures used by Encryption / Decryption 35
7.1.5 Structures used by Seed / Key 36
7.2 Common 36
7.2.1 SecM_InitPowerOn 36
7.2.2 SecM_Task 37
7.3 CRC 37
7.3.1 SecM_ComputeCRC 37
7.4 Encryption / Decryption 38
7.4.1 SecM_InitDecryption 38
7.4.2 SecM_DeinitDecryption 38
7.4.3 SecM_Decryption 39
7.4.4 SecM_InitEncryption 40
7.4.5 SecM_DeinitEncryption 40
7.4.6 SecM_Encryption 41
7.5 Seed / Key Common 42
7.5.1 SecM_GenerateSeed 42
7.5.2 SecM_CompareKey (HIS Specification < 1.0.1) 42
7.5.3 SecM_CompareKey (HIS Specification >= 1.0.1) 43
7.6 Seed / Key default API 44
7.6.1 SecM_ComputeKey 44
7.7 Seed / Key extended API 45
7.7.1 SecM_ComputeKey 45
7.8 Verification 46

7.8.1 SecM_InitVerification 46
7.8.2 SecM_DeinitVerification 46
7.8.3 SecM_Verification 47
7.8.4 SecM_VerifySignature 48

8 Contact 49

Illustrations

Figure 4-1 Example configuration of the SmhGeneral ... 11
Figure 4-2 Example configuration of the SmhJob ... 12
Figure 4-3 Example configuration of a SmhKey ... 13
Figure 4-4 Example configuration of the SmhKeyValue of the SmhCryptographicKey ... 14
Figure 4-5 Example configuration of a copy key element operation ... 14
Figure 4-6 Example configuration of the SmhCrc ... 15
Figure 4-7 Example configuration of the SmhSeedConfig of SmhSeedkey ... 16
Figure 4-8 Example configuration of the SmhKeyConfig of SmhSeedKey ... 16
Figure 4-9 Example configuration of the SmhDecryptionConfig of SmhEncryption ... 17
Figure 4-10 Example configuration of the SmhVerification ... 18
Figure 4-11 Example configuration of the SmhMonolithicVerification ... 19
Figure 4-12 Example configuration of the SmhStreamVerification of SmhVerification ... 20
Figure 6-1 Logical Blocks ... 28
Figure 6-2 Explicit usage of an initialization vector with the encryption API ... 30
Figure 6-3 Explicit usage of an initialization vector with the decryption API ... 31

Tables

Table 2-1 Security Module Components ... 9
Table 3-1 Scope of Delivery ... 10
Table 4-1 Attribute description of SmhCryptographicKeys ... 11
Table 4-2 Attribute description of SmhJob ... 12
Table 4-3 Attribute description of SmhCryptographicKeys ... 12
Table 4-4 Attribute description of SmhKey ... 13
Table 4-5 Attribute description of SmhKeyValue ... 14
Table 4-6 Attribute description of SmhCrc ... 15
Table 4-7 Attribute description of SmhSeedKey ... 15
Table 4-8 Attribute description of SmhSeedConfig ... 16
Table 4-9 Attribute description of SmhKeyConfig ... 17
Table 4-10 Attribute description of SmhEncryption ... 17
Table 4-11 Attribute description of SmhMonolithicVerification ... 20
Table 4-12 Attribute description of SmhStreamVerification ... 21
Table 4-13 Generated files ... 21
Table 5-1 Checksum/signature sizes ... 22
Table 5-2 Format of HMAC key ... 24
Table 5-3 Key structure of the example key ... 24
Table 5-4 Format of public key without additional information ... 24
Table 5-5 Format of public key and private key ... 24
Table 5-6 Key structure of the example key ... 25
Table 5-7 INI-Key file parameter ... 26
Table 6-1 Key handling behavior ... 27
Table 7-1 Type definitions ... 33
Table 7-2 SecM_VerifyParamType ... 34
Table 7-3 SecM_SignatureParamType ... 35
Table 7-4 SecM_CRCParamType ... 35
Table 7-5 SecM_DecInputParamType ... 35
Table 7-6 SecM_DecOutputParamType ... 35
Table 7-7 SecM_DecParamType ... 36
Table 7-8 SecM_KeyType ... 36
Table 7-9 SecM_InitPowerOn ... 36

© 2020 Vector Informatik GmbH

Table 7-10 SecM_Task 37
Table 7-11 SecM_ComputeCRC 37
Table 7-12 SecM_InitDecryption 38
Table 7-13 SecM_DeinitDecryption 38
Table 7-14 SecM_Decryption 39
Table 7-15 SecM_InitEncryption 40
Table 7-16 SecM_DeinitEncryption 40
Table 7-17 SecM_Encryption 41
Table 7-18 SecM_GenerateSeed 42
Table 7-19 SecM_CompareKey 43
Table 7-20 SecM_CompareKey 43
Table 7-21 SecM_ComputeKey 44
Table 7-22 SecM_ComputeKey 45
Table 7-23 SecM_InitVerification 46
Table 7-24 SecM_DeinitVerification 46
Table 7-25 SecM_Verification 47
Table 7-26 SecM_VerifySignature 48

1 Introduction

This document describes the interface and configuration of the security module. The main goal of this component is to provide the following functionalities during the ECU flash operations:

- Authorization
- Authentication, verification and error detection
- Confidentiality

The **authorization** part of the security component can be used for the Seed/Key procedure to unlock an ECU. The module provides a standard interface for both, seed and key generation.

The functions are available in source code and can be adapted if needed.

Authentication, verification and error detection is available in three different levels. They are distinguished by so-called security classes:

- Class DDD (Hash, CRC)
- Class C (MAC)
- Class CCC (Signature)

Only one of the three classes can be selected at a time and differs in the strength of the mathematical algorithm.

The security classes C and CCC calculate a signature on one or more data segments. The resulting signature is compared against a signature that is given through the flash download. The checksum calculation with the security class DDD has in principle the same mechanism.

A cryptographic key (symmetric or asymmetric) is required for security classes C and CCC algorithm calculation but is not required for security class DDD, which makes use of simple checksum/hash algorithms.

The given signatures must be calculated off-line on exactly the same set of data that the security component uses for verification. For security class C, the cryptographic key for the on-line and off-line signature is the same. This is called a symmetric signature calculation.

For security class CCC, an asymmetric signature calculation is used. The off-line signature is calculated with a private key, and the on-line signature calculation (respectively the verification) uses the public key.

The length of the signature varies depending on the used security classes.

Confidentiality can be achieved by data encryption and decryption (security class AAA). The security module offers an interface for symmetrical encryption / decryption.

The security classes and APIs are standardized by the HIS (see [2]).

© 2020 Vector Informatik GmbH

2 Software Architecture

The Security Module delivery contains the following components:

- The security module itself
- Configuration tool DaVinci Configurator 5

In addition, the following tool is delivered:

- The software tool HexView. It can be used to change and show the contents of a binary file. Also, it can be used for signature and checksum calculation. Besides, the creation of encrypted download files is possible with HexView.

2.1 Security Module Components

The functionalities of the Security Module are separated into several sub-modules. A functional description for the individual modules can be found in chapter 6.

The following modules are provided with the Security Module:

Name	Description
CRC	Implementation of the HIS security module - CRC calculation. Offers CRC calculation according to CRC-16 CCITT, CRC-32 (IEEE 802.3) and CRC-64.
Decryption	Implementation of the HIS security module – Decryption. Offers API for decryption and encryption.
SeedKey	Implementation of the HIS security module - Seed/key authentication. Offers challenge/response authentication interface.
Verification	Implementation of the verification component of the HIS security module - Signature verification. Offers signature/checksum verification interface.
Job	Implementation of the interface to the AUTOSAR CSM module.
Sec	Implementation of the HIS security module.

Table 2-1 Security Module Components

3 Integration

This chapter gives necessary information for the integration of the Vector Security Module into an application environment of an ECU.

3.1 Scope of Delivery

The delivery of the Security Module contains the following files:

Module	Filename	Description
CRC	Sec_Crc.c	Source file of the CRC Module.
	Sec_Crc.h	Header file of the CRC Module.
Decryption	Sec_Decryption.c	Source file of the Decryption Module.
	Sec_Decryption.h	Header file of the Decryption Module.
SeedKey	Sec_SeedKey.c	Source file of the Seed/Key Module.
	Sec_SeedKey.h	Header file of the Seed/Key Module.
Verification	Sec_Verification.c	Source file of the Verification Module.
	Sec_Verification.h	Header file of the Verification Module.
Common	Sec_Inc.h	Header file used internally.
	Sec_Types.h	Defines types, constantans and configuration switches.
	Sec.c	Source file of the Security Module.
	Sec.h	Header file of the Security Module.
	Sec_Job.c	Source file of the CSM abstraction layer.
	Sec_Job.h	Source file of the CSM abstraction layer.
	Sec_Compat.h	Header file for backward compatibility.
	SecM_Inc.h	Header file for backward compatibility.
	SecM.h	Header file for backward compatibility.

Table 3-1 Scope of Delivery

3.2 Include Structure

For usage of the Security Module within an application it is necessary to include the ‘Sec.h’ header file.

4 Configuration

A list of all attributes and a small description are provided in each section. For more details of the attributes, please refer to the description in DaVinci Configurator 5.

4.1 General

This container is used to make global/general configuration for the SMH module.

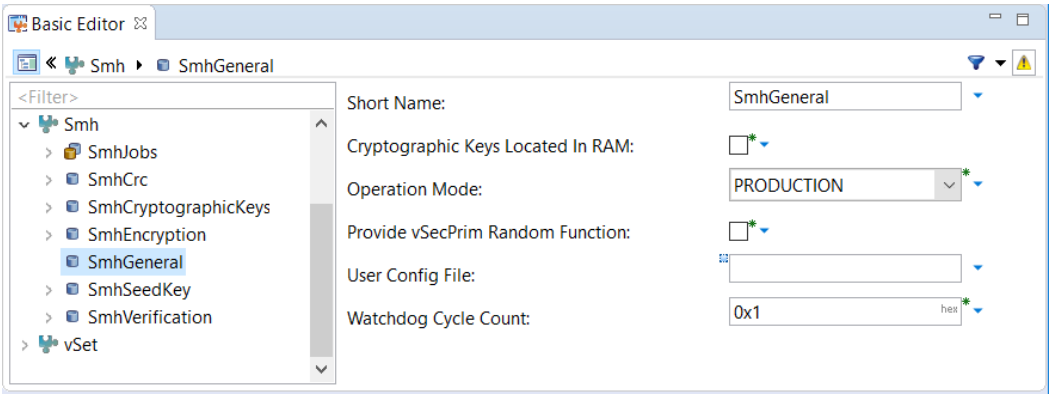


Figure 4-1 Example configuration of the SmhGeneral

Attribute	Description
SmhCryptographicKeysLocatedInRAM	Specify the location of the SmhCryptographicKeys
SmhOperationMode	Specify the operation mode
SmhProvidevSecPrimRandomFunction	Generates the function esl_getBytesRNG required by vSecPrim
SmhUserConfigFile	Path to a file, where the content is placed into the SecM_Cfg.h file
SmhWatchdogCycleCount	Specify the cycle count for calling the watchdog function

Table 4-1 Attribute description of SmhCryptographicKeys

4.2 Job

Similar to the job configuration of the CSM module (see [7]), it is also required to configure for the security module a so-called SmhJob. A SmhJob contains different information depending on the selected sub-container. The job type is determined by a sub-container. The sub-container usually contains a reference to a CsmJob and a reference to a cryptographic key (see chapter 4.3).

If a SmhJob does not reference a cryptographic key, the value currently present in the crypto module is used for the job. This could be either the default value of the respective key element in the crypto module or an externally modified value, e.g. from a different job. If the cryptographic key is referenced in the SmhJob, the key value defined in the container is used. Please see chapter 6.1 for more information.

An example job with job type “SmhDecryption” is shown in Figure 4-2.

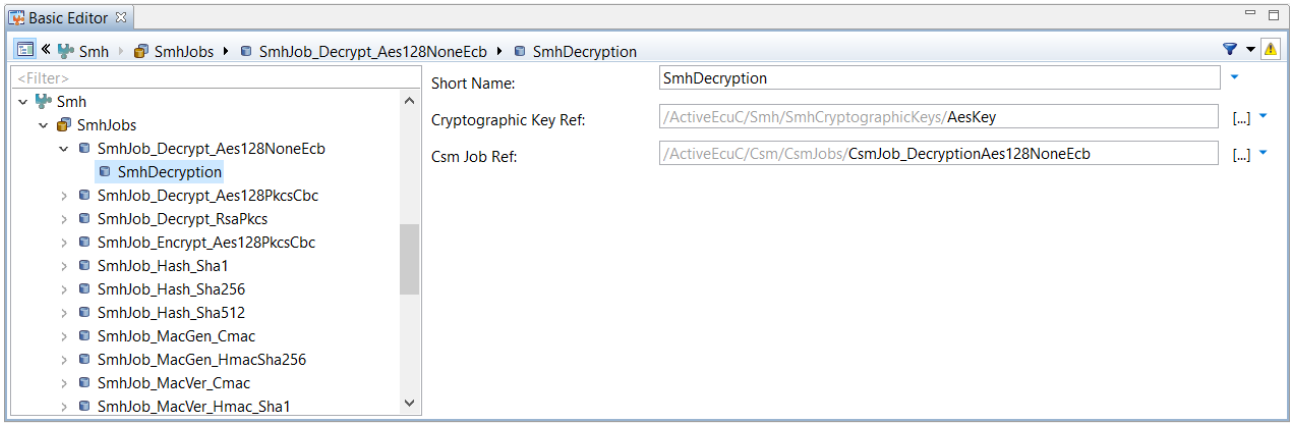


Figure 4-2 Example configuration of the SmhJob

SmhJob type	Attributes	Description
SmhDecryption	SmhCryptographicKeyRef	Reference to a SmhKey (optional)
	SmhCsmJobRef	Reference to a CsmJob
SmhEncryption	SmhCryptographicKeyRef	Reference to a SmhKey (optional)
	SmhCsmJobRef	Reference to a CsmJob
SmhHash	SmhCsmJobRef	Reference to a CsmJob
SmhMacGenerate	SmhCryptographicKeyRef	Reference to a SmhKey (optional)
	SmhCsmJobRef	Reference to a CsmJob
SmhMacVerify	SmhCryptographicKeyRef	Reference to a SmhKey (optional)
	SmhCsmJobRef	Reference to a CsmJob
SmhRandomGenerate	SmhCsmJobRef	Reference to a CsmJob
SmhRandomSeed	SmhCsmKeyRef	Reference to a CsmKey
SmhSignatureVerify	SmhCryptographicKeyRef	Reference to a SmhKey (optional)
	SmhCsmJobRef	Reference to a CsmJob

Table 4-2 Attribute description of SmhJob

4.3 Cryptographic Keys

A cryptographic key consists of a key and key value container.

Also, it is required to specify the key element ID for the RSA modulus and the RSA public exponent.

Attribute	Description
SmhElementIdRSAModulus	Key element ID of the RSA modulus
SmhElementIdRSAPublicExponent	Key element ID of the RSA public exponent

Table 4-3 Attribute description of SmhCryptographicKeys

4.3.1 Key

The key container is used to assign several key values to the key.

Figure 4-3 shows an example for an RSA key. The modulus and public exponent key values are referenced.

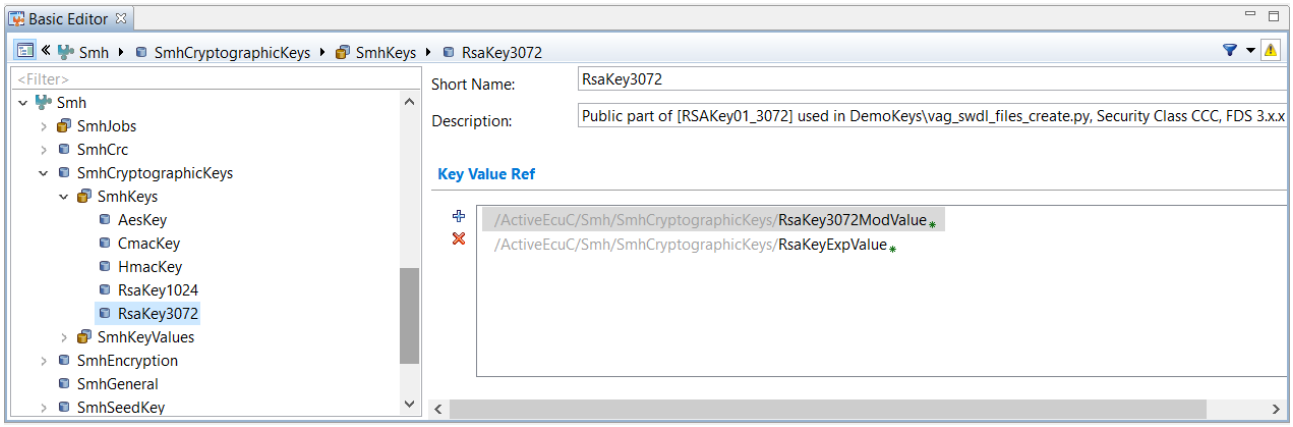


Figure 4-3 Example configuration of a SmhKey

Attribute	Description
SmhDescription	Short description of the key
SmhKeyValueRef	List of assigned key values.

Table 4-4 Attribute description of SmhKey

4.3.2 Key Values

A KeyValue container is used to configure an explicit key. The key value contains two important parameters:

- Crypto key element reference
- (actual) value

The crypto reference is used to determine the key element, which can be updated with an internal or external key.

The value field shall contain the value of the internal key. If the field is left empty, no internal key value is known and only an external key can be used to update the crypto key element. For more information, please refer to chapter 6.1.

Figure 4-4 shows an example for the IV value of an AES decryption, with empty value field. This means that the IV value needs to be provided externally.

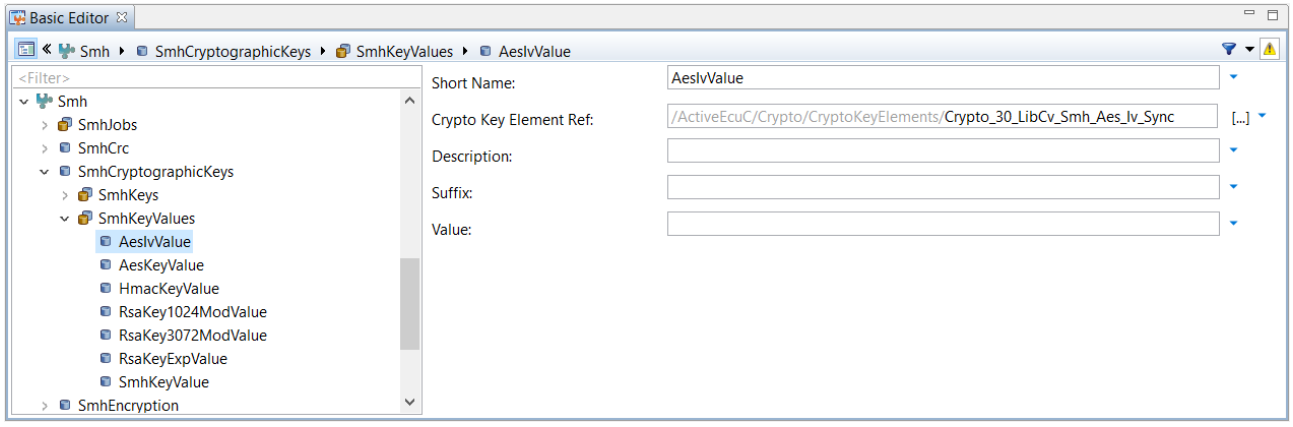


Figure 4-4 Example configuration of the SmhKeyValue of the SmhCryptographicKey

Attribute	Description
SmhCryptoKeyElementRef	Reference to the CryptoKeyElement
SmhDescription	Short description of the key value
SmhSuffix	Suffix of the key value
SmhValue	Actual value of the key

Table 4-5 Attribute description of SmhKeyValue

4.3.3 Key Operations

With key operations it is possible to generate additional macros, which allow to perform the following operations with configured cryptographic keys

- Access Key Element: Get or Set the specified key element
- Copy Key Element: Copy a single key element from source to target
- Copy Key: Copy a complete key, including all included key elements, from source to target.

Figure 4-5 shows an example configuration for a Copy Key Element operation.

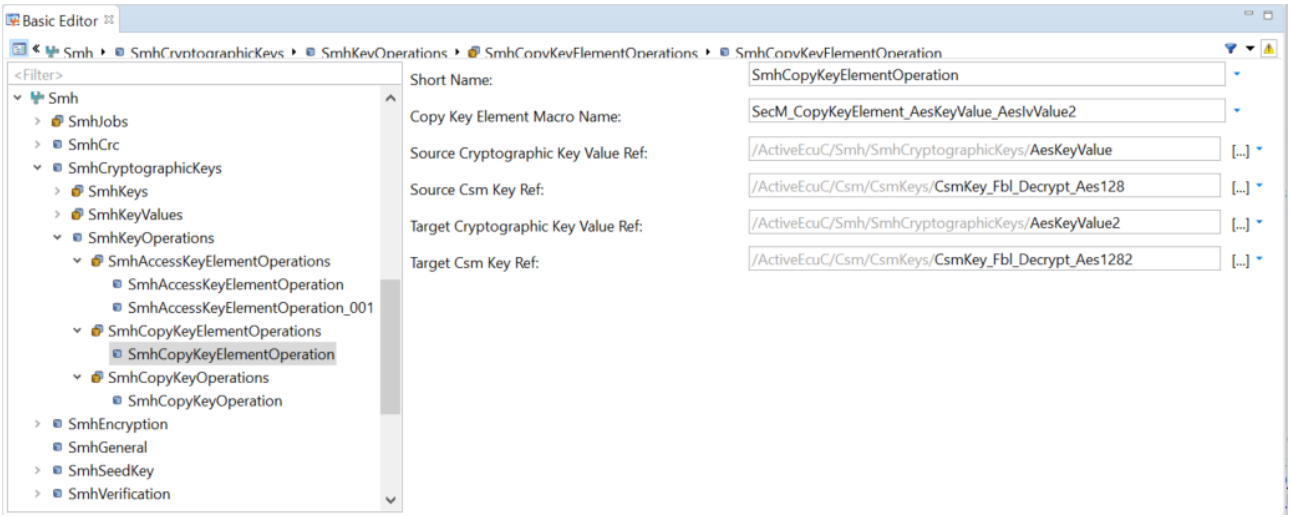


Figure 4-5 Example configuration of a copy key element operation

4.4 CRC

The CRC module provides three algorithms (CRC-16, CRC-32 and CRC-64), which are enabled by the respective sub-container. Each CRC sub-container can be configured independently.

The CRC type selects the default CRC algorithm. This default CRC will be used for the CRCTotal and CRCWritten. Furthermore, it will be used by the function SecM_ComputeCRC.

Figure 4-6 shows an example configuration of the CRC-32.

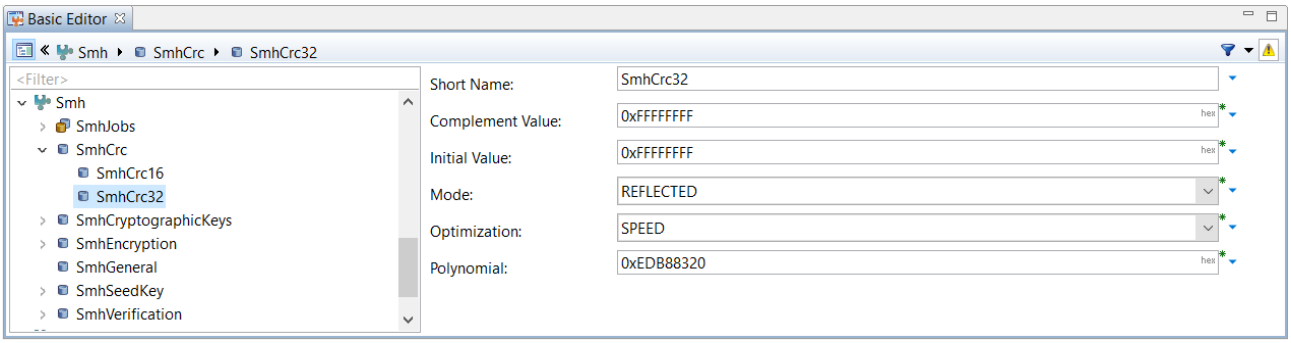


Figure 4-6 Example configuration of the SmhCrc

Sub-Container	Attribute	Description
SmhCrc	SmhCRCType	Selects the default CRC algorithm
SmhCrc[16, 32, 64]	SmhComplementValue	Finalization value for CRC
	SmhInitialValue	Initialization value for CRC
	SmhMode	Operation mode of the CRC calculation
	SmhOptimization	Select the optimization of the CRC calculation
	SmhPolynomial	Polynomial of the CRC calculation

Table 4-6 Attribute description of SmhCrc

4.5 Seed/Key

The seed/key configuration consists of the seed and key sub-containers.

Attribute	Description
SmhCallCycle	Specifies the call cycle
SmhKeyTimeout	Currently not used

Table 4-7 Attribute description of SmhSeedKey

4.5.1 Seed

The seed generation supports two algorithms, which can be selected via the choice container:

> LCG (internal “linear congruential generator” algorithm)

> CSM (uses the CSM module to generate a random number)

If a CSM is used to generate a seed, an entropy is required. The entropy configuration needs to be added.

An example configuration, which uses the CSM and an entropy is shown in Figure 4-7.

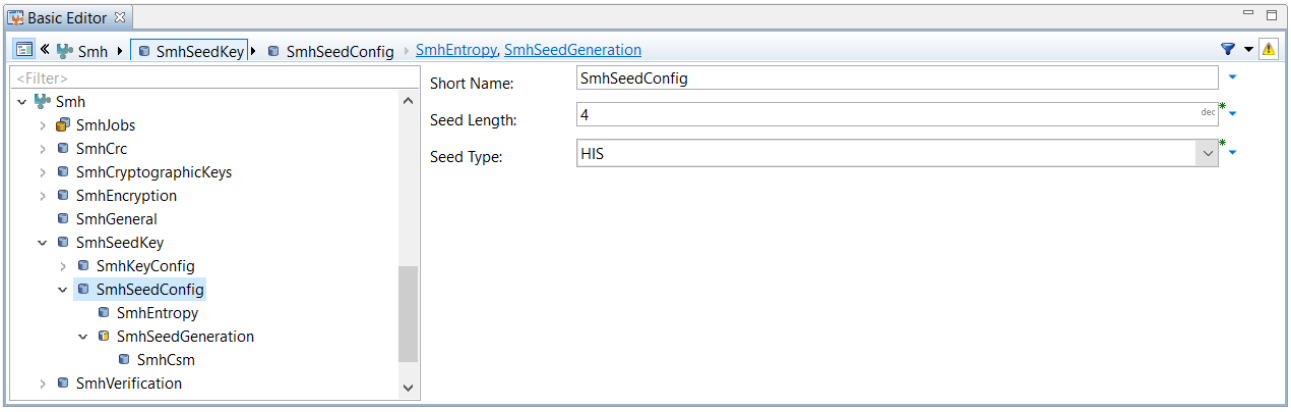


Figure 4-7 Example configuration of the SmhSeedConfig of SmhSeedkey

Sub-Container	Attribute	Description
SmhSeedConfig	SmhSeedLength	Specify the length of the seed in bytes
	SmhSeedType	Specify the data structure of the seed
SmhEntropy	SmhEntropySize	Size of the used entropy
	SmhEntropyUsage	Specify the usage of the entropy
	SmhJobRef	Reference to a SmhJob
SmhSeedGeneration:SmhCsm	SmhJobRef	Reference to a SmhJob
SmhSeedGeneration:SmhLcg	SmhIncrement	Increment of the LCG algorithm
	SmhMultiplier	Multiplier of the LCG algorithm

Table 4-8 Attribute description of SmhSeedConfig

4.5.2 Key

Defines the required jobs to compute the key. This will create easy to use macros/functions to execute specific jobs.

Figure 4-8 shows a key configuration with two SmhKeyJobs. These jobs can be used during the key computation.

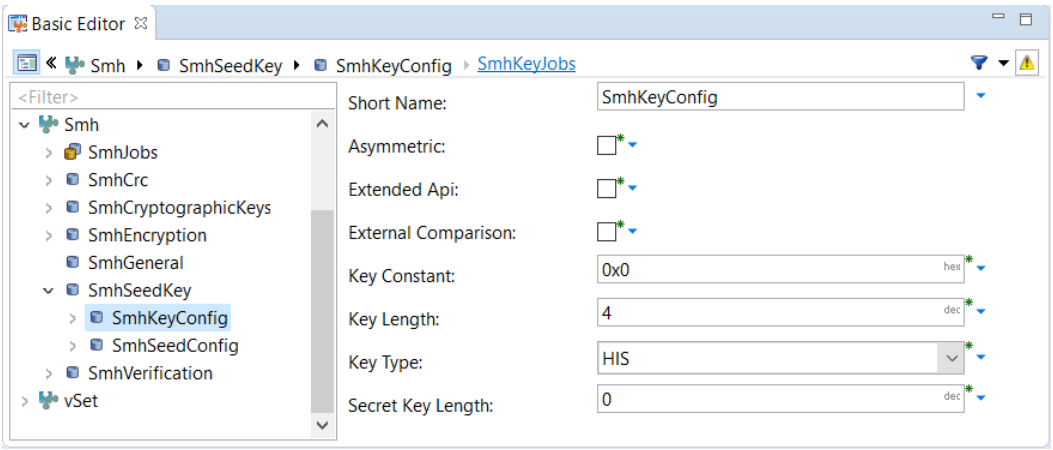


Figure 4-8 Example configuration of the SmhKeyConfig of SmhSeedKey

Sub-Container	Attribute	Description
SmhKeyConfig	SmhAsymmetric	Enables the asymmetric key computation
	SmhExtendedApi	Enables the extended API
	SmhExternalComparison	Use an external function to compare the key
	SmhKeyConstant	Key constant (ECU key)
	SmhKeyLength	Specify the length of the key in bytes
	SmhKeyType	Specify the data structure of the key
	SmhSecretKeyLength	Specify the length of the secret key in bytes
SmhKeyJobs	SmhFunctionName	Specify the name of the function
	SmhJobRef	Reference to a SmhJob

Table 4-9 Attribute description of SmhKeyConfig

4.6 Encryption/decryption

This container references several encryption/decryption jobs, which can be used during the download or during other services.

The following Figure 4-9 shows an example configuration.

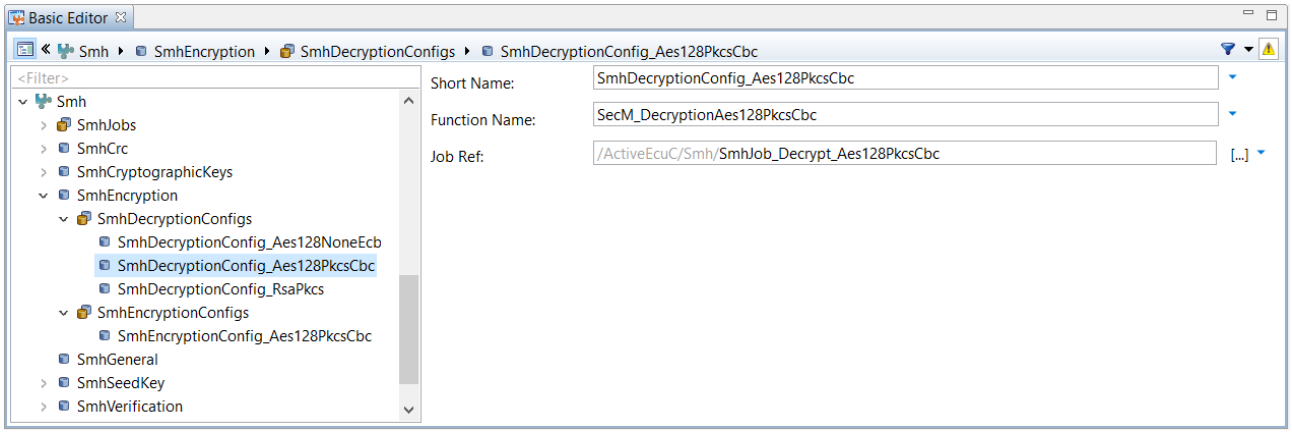


Figure 4-9 Example configuration of the SmhDecryptionConfig of SmhEncryption

Sub-Container	Attribute	Description
SmhEncryption	SmhDecryptionDefaultConfig	Specify the default decryption (used by SecM_Decryption)
	SmhEncryptionDefaultConfig	Specify the default encryption (used by SecM_Encryption)
SmhDecryptionConfig	SmhFunctionName	Specify the name of the function
	SmhJobRef	Reference to a SmhJob
SmhEncryptionConfig	SmhFunctionName	Specify the name of the function
	SmhJobRef	Reference to a SmhJob

Table 4-10 Attribute description of SmhEncryption

4.7 Verification

This container is used to configure the different Security Classes:

- DDD
- C
- CCC
- Vendor

Vendor is a special security class in case the default algorithms are not sufficient to support the OEM (or customer) requirements. It can also be used by the customer to provide external algorithms.

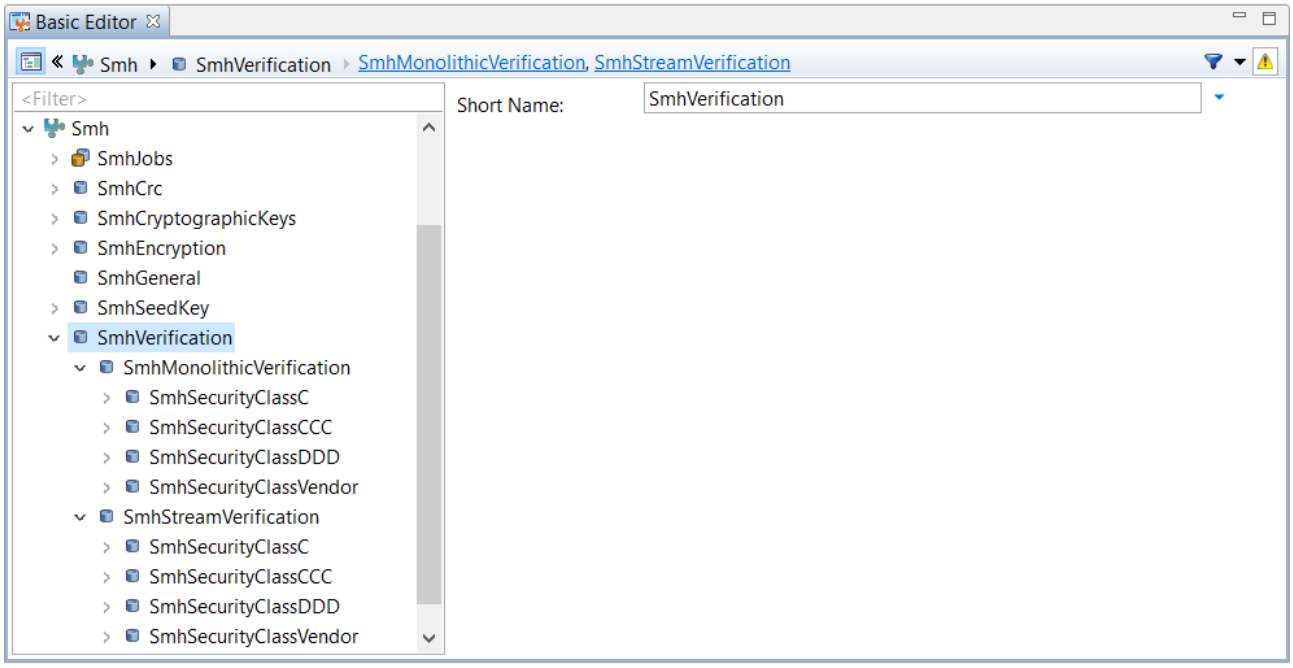


Figure 4-10 Example configuration of the SmhVerification

4.7.1 Monolithic Verification

The monolithic verification is used for the “output” verification (as defined in the logical block table) of the Bootloader but can also be used in other contexts if required.

The monolithic verification includes the following verification operations depending on the configuration:

- Normal stream verification. Optionally, the address and length can be included into the signature.
- Calculates the CRC Total, if the feature is enabled. For more information about CRC Total, please refer to chapter 6.3.2.
- Perform the vendor verification for the downloaded segments, if the reference to a vendor verification is selected.
- Calculate an additional CRC, also called CRC Written, over the downloaded segments and verify it, if the feature is enabled.

> Return the above-mentioned CRC Written to the calling layer instead of verifying it

Figure 4-11 shows an example configuration of a Security Class CCC configuration.

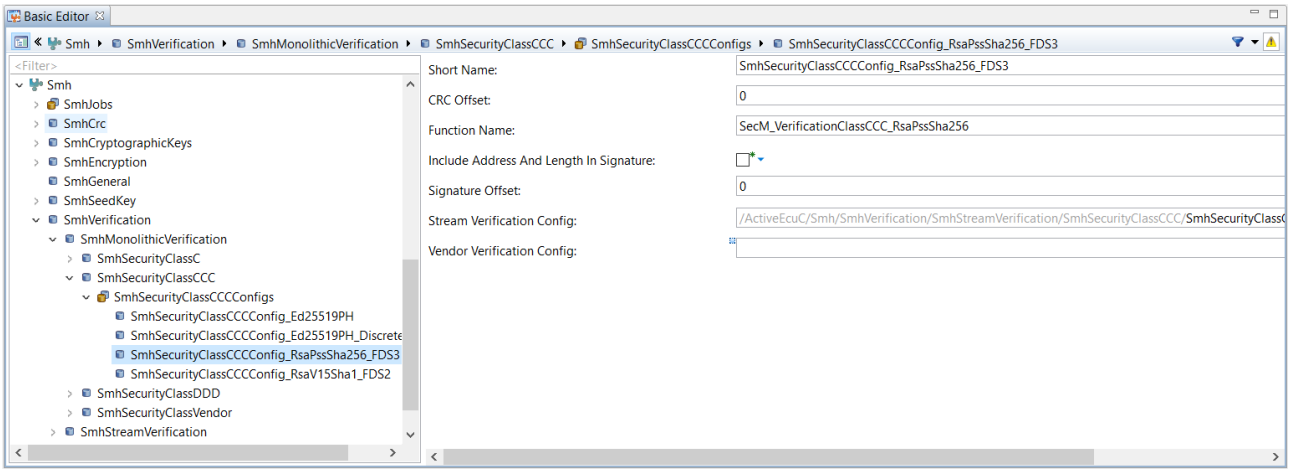


Figure 4-11 Example configuration of the SmhMonolithicVerification

Sub-Container	Attribute	Description
SmhMonolithic Verification	SmhAssertReadCount	Checks that the read count is as expected
	SmhCRCTotal	Calculates a CRC over the complete logical block
	SmhCRCWritten	Calculates and verifies an additional CRC over the segments
	SmhCRCWrittenGet	Return the CRC Written to the calling layer instead of verifying it
	SmhCheckAddressRange Overflow	Check that there is now address range overflow
	SmhDefaultMonolithic VerificationConfig	Specify the default monolithic verification (used by SecM_Verification)
	SmhExportVerificationWrapper	Exports the function SecM_VerificationWrapper
	SmhMemoryAccessMode	Currently not used, fixed to ADDRESS
	SmhOperationCountOfWrapper	Specify the number of operations
	SmhReadBufferSize	Specify the length of the read buffer
SmhSecurityClassDDD	SmhDefaultSecurityClass DDDConfig	Default monolithic verification used by SecM_VerificationClassDDD
SmhSecurityClassC	SmhDefaultSecurityClass CConfig	Default monolithic verification used by SecM_VerificationClassC
SmhSecurityClassCCC	SmhDefaultSecurityClass CCCConfig	Default monolithic verification used by SecM_VerificationClassCCC
SmhSecurityClassVendor	SmhDefaultSecurityClass VendorConfig	Default monolithic verification used by SecM_VerificationClassVendor

Sub-Container	Attribute	Description
SmhSecurityClass[DDD, C, CCC, Vendor]Config	SmhCRCOffset	Offset to CRC Written
	SmhFunctionName	Specify the name of the function
	SmhIncludeAddressAndLengthInSignature	Include the address and length of the segments into the signature
	SmhSignatureOffset	Specify the offset to the signature
	SmhStreamVerificationConfig	Reference to the underlying StreamVerification
	SmhVendorVerificationConfig	Reference to a VendorVerification

Table 4-11 Attribute description of SmhMonolithicVerification

4.7.2 Stream Verification

The stream verification is referenced by the monolithic verification. It contains a specific algorithm (e.g. RSA, HMAC, CRC, …) to verify data with a stream-based interface (start, update, finish).

The stream interface is used for the “input”, “processed” and “pipelined” verification (as defined in the logical block table) of the Bootloader but can also be used in other contexts if required.

The Security Class CCC container contains also a sub-container to configure a discrete hash. The data is first hashed and the hash value is then verified using the respective signature algorithm.

An example configuration of a Security Class CCC is shown in Figure 4-12.

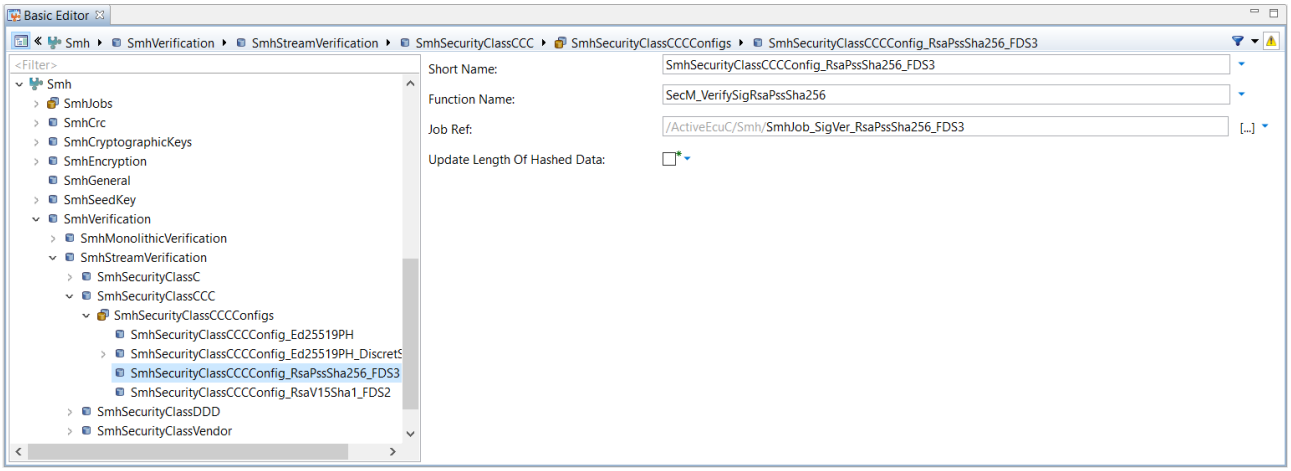


Figure 4-12 Example configuration of the SmhStreamVerification of SmhVerification

Sub-Container	Attribute	Description
SmhStream Verification	SmhDefaultStream VerificationConfig	Specify the default stream verification (used by SecM_VerifySignature)
SmhSecurityClassDDD	SmhDefaultSecurityClass DDDConfig	Default stream verification used by SecM_VerifyClassDDD
	SmhComputationOnly	Compute the only CRC/hash

Sub-Container	Attribute	Description
SmhSecurityClass DDDConfig	SmhFunctionName	Specify the name of the function
	SmhUpdateLengthOf HashedData	Update the current length of hashed data
SmhSecurityClassDDD Type:Crc	SmhCrcRef	Reference to a SmhCrcXX container
SmhSecurityClassDDD Type:Hash	SmhJobRef	Reference to a SmhJob
SmhSecurityClassC	SmhDefaultSecurityClass CConfig	Default stream verification used by SecM_VerifyClassC
SmhSecurityClass CConfig	SmhComputationOnly	Compute only the MAC
	SmhFunctionName	Specify the name of the function
	SmhJobRef	Reference to a SmhJob
	SmhUpdateLengthOf HashedData	Update the current length of hashed data
SmhSecurityClassCCC	SmhDefaultSecurityClass CCCConfig	Default stream verification used by SecM_VerifyClassCCC
SmhSecurityClass CCCConfig	SmhFunctionName	Specify the name of the function
	SmhJobRef	Reference to a SmhJob
	SmhUpdateLengthOf HashedData	Update the current length of hashed data
SmhDiscretHashConfig	SmhJobRef	Reference to a SmhJob
SmhSecurityClassVendor	SmhDefaultSecurityClass VendorConfig	Default stream verification used by SecM_VerifyClassVendor
SmhSecurityClass VendorConfig	SmhFunctionName	Specify the name of the function
	SmhSizeOfVendorChecksum	Specify the size of the vendor checksum

Table 4-12 Attribute description of SmhStreamVerification

4.8 Configuration files

The following table describes the contents of the generated files:

File Name	Description
SecM_cfg.h	The configuration file of the security module. For the most part the file defines switches in the form of SEC_ENABLE_<feature> or SEC_DISABLE_<feature> according to the settings in the dialog.
SecMPar.c, SecMPar.h	This file contains configured keys used by encryption/decryption as well as for verification and the primitives required for monolithic/stream verification.

Table 4-13 Generated files

5 Preparation of the Application

Before a download can be made preparing the download file is a necessary step. For bootloaders with logical block tables, the address space used by one single file has to match information of a single block stored in the logical block table. It is possible to download multi region files, but the region count is limited by the Bootloader.

Each download file requires a signature or resp. a checksum file, which is created out of the file contents. A signature file is a byte stream in hexadecimal format with comma separation (e.g. 0x12, 0x34, 0x45). The number of bytes within a signature file and its creation depend on the security class that is used for the download.

The following three different security classes are distinguished (see chapter 1):

- Security Class DDD
- Security Class C
- Security Class CCC

The following table gives a short overview of some algorithms and their checksum/signature size:

Security Class	Algorithm	Checksum/signature size
DDD	CRC-16	2
	CRC-32	4
	CRC-64	8
	SHA-1	20
	SHA-224	28
	SHA-256	32
	SHA-384	48
	SHA-512	64
	RIPEMD-160	20
C	CMAC	16
	HMAC	Refer to the hash sizes of SHA and RIPEMD
CCC	RSA-512	64
	RSA-1024	128
	RSA-1536	192
	RSA-2048	256
	RSA-3072	384
	RSA-4096	512
	ECDSA	64

Table 5-1 Checksum/signature sizes

The key information for security class C and CCC are required by a tool (HexView) to create this signature from the download file. There are two file formats (see chapter 5.1), which can be used for HexView.



Note
The keys for series production are usually provided by the OEM. However, to support the development process, some sample keys are part of the Bootloader delivery (see folder .\Demo\DemoKeys in the delivery). They are NOT INTENDED for series production, but only for development purposes.

For class C the same secret key information is needed by the Bootloader for the verification process after a successful flash download. Thus, the contents of the secret key must be stored in the generation tool.

Store the secret key carefully away and do not provide this information to others, as this is the secret for further updates of your ECU software.

The signature file for the Security Class CCC is depending on the key length of the chosen RSA (see Table 5-1). It uses a public and a private key, similar to PGP. The private key is necessary to create the signature files, whereas the public key is used for the verification process within the Bootloader. It is necessary to use the private key to create the signature files.

The key files for Security Class CCC are also stored in the ASN.1 format. The creation of the verification data through the generation tool and the signature files through HexView are similar to the Security Class C.

To support you in the development process, the Bootloader product comes along with some tools to support you to create these checksum and signature files.

5.1 Key Files

The key files can be provided to HexView in two different formats:

- Authenticity file
- INI-Key file

5.1.1 Authenticity file

The authenticity file is a simply one-line text file with a hex byte stream in a special format (pseudo-ASN.1), which has been specified in the see [2] HIS Security Module Specification.

Security Class C:

Tag	Length description	Length (byte)	Value
‘FF 59’	‘XX’ resp. ’81 XX’	max. 34	Object with key components of a private key
‘D3’	’XX’ resp. ‘81’ XX typically ‘14’	xx resp. 20	HMAC key

Table 5-2 Format of HMAC key



Example
FF5916D3145F1CBE397C4AF8956E26DC4DAED95DB25A14B429

Tag	Description
‘FF 59’	Tag: Private key
‘16’	Total length information
‘D3’	Tag: HMAC key
‘14’	Length of key data
‘5F…29’	Key data

Table 5-3 Key structure of the example key

Security Class CCC:

Object with key components of a public key:

Tag	Length description	Length (byte)	Value
‘7F 49’	’81 XX’ ’82 XX XX’	-	Object with key components of a public key
‘81’	’81 80’ resp. ’82 01 00’	128 resp. 256	Modulus ‘n’
‘82’	XX	typically 2…3	Public (encryption) exponent ‘e’

Table 5-4 Format of public key without additional information

Object with key components of a public and a private key:

Tag	Length description	Length (byte)	Value
‘FF 49’	’81 XX’ resp. ’82 XX XX’	-	Object with key components of a public and private key
‘81’	’81 80’ resp. ’82 01 00’	128 resp. 256	Modulus ‘n’
‘82’	XX	typically 2…3	Public (encryption) exponent ‘e’
‘91’	’81 80’ resp. ’82 01 00’	128 resp. 256	Private (decryption) exponent ‘d’

Table 5-5 Format of public key and private key

© 2020 Vector Informatik GmbH



Example
FF49 82 010B 81 81 80 D7D9F92AF1E4DD67DABC9169E76F880D49CE9707BA95
CFDBE3C6F1D8AAEE2887F04F3F1447DBDF66F70AE282D8701F74D65281203413
F86E0FD3014FF3D80099C01FBEE2E4316EEB64A6412D285AC52010762BD79994
4A80333A74B4B59164773291F26A162680E61A764764DC04FC86C133CCF008A09
AF601434C4025C15507 82 03 010001 91 81 80 4AD2D69ADA6B598CACB87D37D
6F0449B46A0849E3B41BF621C1D54ACA4E415CCB9B2DECCEAF3FF07344FE3A1
33E1FA25883DE561026C81CBB55491B9E36F571551DA6B2D1FF42A03BD5F5DDF
2FCA2F3BC3EF9F409A6B9B5E91C80811AB80B294A0A3ED9CA122255EF5C0CEC
DEBD2652A613B2DB6AFA588D71B42005528FAB231

Tag	Description
‘FF 49’	Tag: Private key
‘82’	2 byte length information (length >= 0x80)
’01 0B’	Total length
‘81’	Tag: Modulus ‘n’
‘81’	1 byte length information (length >= 0x80)
‘80’	Length of modulus
‘D7 … 07’	Modulus ‘n’
‘82’	Tag: Public exponent ‘e’
‘03’	Length of exponent (length < 0x80; no additional length information)
’01 00 01’	Public exponent ‘e’
‘91’	Tag: Private exponent ‘d’
‘81’	1 byte length information (length >= 0x80)
‘80’	Length of exponent
‘4A … 31’	Private exponent ‘d’

Table 5-6 Key structure of the example key

5.1.2 INI-Key file mode

The INI-Key mode is a proper way to manage the keys. The key is also based on the ASN.1 format (see 5.1.1) but this file can be used to provide additional information and can include multiple keys.

Parameter	Description
DESC	Description text which is showed in GENy.
VALUE (C) or KEY-PRIV (CCC)	The key in ASN.1 format.
ECU	ECU identifier.
CREATION-DATE	Timestamp of the creation of the key
PURPOSE	Describes the purpose of the key being used. E.g. production, development …

Parameter	Description
SECURITY-CLASS	Information about the security class.

Table 5-7 INI-Key file parameter

An example can be seen below:

```
[RSAKey01]

DESC = 'This is the development key.'

KEY-PRIV =

'FF4982010B818180D7D9F92AF1E4DD67DABC9169E76F880D49CE9707BA95CFDBE3C6F1D8AAEE288
7F04F3F1447DBDF66F70AE282D8701F74D65281203413F86E0FD3014FF3D80099C01FBEE2E4316EE
B64A6412D285AC52010762BD799944A80333A74B4B59164773291F26A162680E61A764764DC04FC8
6C133CCF008A09AF601434C4025C155078203010001918180 4AD2D69ADA6B598CACB87D37D6F0449
B46A0849E3B41BF621C1D54ACA4E415CCB9B2DECCEAF3FF07344FE3A133E1FA25883DE561026C81C
BB55491B9E36F571551DA6B2D1FF42A03BD5F5DDF2FCA2F3BC3EF9F409A6B9B5E91C80811AB80B29
4A0A3ED9CA122255EF5C0CECDEBD2652A613B2DB6AFA588D71B42005528FAB231'

ECU = My ECU

CREATION-DATE = 2006-04-13T12:02:09+02:00

PURPOSE = Development

SECURITY-CLASS = CCC
```

5.2 HexView

HexView is a software tool from Vector Informatik. It can be used to change and show the contents of several file formats (e.g. binary, Intel-Hex, Motorola S-Record, OEM container, …). For more information about HexView, please refer to see [3] HexView Reference Manual.

6 Functional Description

This chapter describes the security modules, their variants and provides implementation samples for some common use-cases.

6.1 Key handling

Depending on the configuration of the SmhJob (see chapter 4.2) and the configured SmhKey (see chapter 4.3) the security module will update the CryptoKeyElement with the external key, the internal key or no update of the CryptoKeyElement is done. For a detailed description, please refer to Table 6-1.

Key referenced in SmhJob	Value present in SmhKeyValue	Behavior
Yes	Yes	Update the key element with the external key, if provided. If not, use the internal key.
Yes	No	Update the key element with the external key, if provided. if not, the key will not be updated and the current key present in the crypto module will be used.
No	-	The key will not be updated and the current key present in the crypto module will be used.

Table 6-1 Key handling behavior

6.2 Verification Module

In the Bootloader case the security component is used to verify one or more sections of data. Depending on the security class, it consists of more or less complex mathematical routines that create a set of data with fixed length out of these sections of data. This fixed length data is called the signature or checksum.

It is specific to the mathematical routines that the signature changes drastically even though just a small portion or even a bit in the data stream has changed. It is also specific, that a key is necessary to create this signature.

6.2.1 Description of the verification process

As mentioned above, the signature is created on a set of data. This is usually a data stream that shall be downloaded and programmed into the memory of an ECU.

For this purpose, the flash memory is partitioned into logical blocks. Each logical block can store a program or other code that consists of one or more sections.

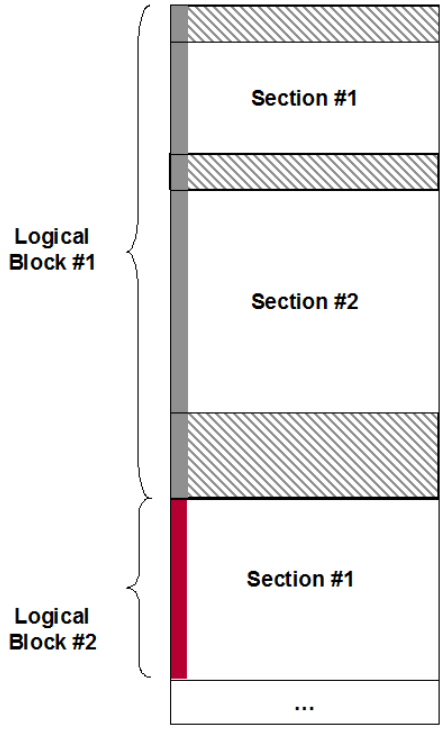


Figure 6-1 Logical Blocks

The checksum/signature is created over the segments of a logical block. Thus, for the module of logical block #1, there are two segments available.

Usually the signature is not only generated on the data itself. The address and length information of each segment is also added to the creation of the signature.

The segments reside in physical address regions. However, it can be necessary, that the address information for the download does not correspond to the physical addresses, because of necessary address translations during the download, e.g. for banked systems or re-locatable modules. Here, logical or virtual addresses are used instead of physical addresses. Note that the signature will be based only on the logical or virtual address information, because only this information is available when creating the signature. However, when the data shall be read from the memory, the physical address information is required.

As a result, the SecM_Verification process requires both the address information that was given by the download and the address information where the data resides in memory. The logical or virtual address information is stored in the transferredAddress element of the segment definition.

6.3 CRC Module

The CRC is an error-detecting code, which can be used to build a checksum over a defined segment. If this module was ordered a 16, 32 or 64 Bit CRC implementation is included in the delivery. The SecM_ComputeCRC function can be called multiple times. The so-called streaming approach can be used for the verification of a data stream. It can be necessary to verify the integrity of a logical block.

6.3.1 Variants

> CRC-16 (CCITT),CRC-32 (IEEE 802.3) or CRC-64

> CRCTotal

> Speed/Size optimized

6.3.2 CRCTotal

To avoid storing the whole segment information with the resulting signature, the security module calculates a CRCTotal. This CRCTotal is generated internally by the security component during the verification process by the function SecM_Verification(). It is only necessary to pass the blockStartAddress and the blockLength of the logical block. The results will be placed into the CRCtotal field by the security module. If the CRCTotal option is activated in GENy it is always calculated, regardless of the security class.



Note
The CRCWritten is the CRC-value as the signature value calculated over the segments for the security class DDD.

6.3.3 Speed / size optimization

In the configuration the CRC module can be switched between two different modes for each CRC algorithm: Speed or size optimized. When ‘SPEED optimized’ is chosen, a look-up table will be generated.



Caution
If the checksum calculation shall also be used within the application, we strongly recommend to use ‘SIZE optimized’

6.3.4 Operation mode flags

These operation mode flags are valid and can be set by using the ‘crcState’.

> SEC_CRC_INIT

Sets the initial CRC value and initialize the look-up table.

> SEC_CRC_COMPUTE

Update the CRC calculation using the provided data.

> SEC_CRC_FINALIZE

Optional: Complete the CRC value

6.4 Encryption / Decryption Module

The Encryption / Decryption Module provide encryption and decryption of data using AES..

6.4.1 Variants

> AES 128/192/256 bit encryption / decryption in CBC/ECB mode

> With or without initialization vector

6.4.2 Using an explicit initialization vector

The first 16 bytes (block size) of data will be interpreted as the initialization vector, when the operation mode flag SEC_DECRYPTION_MODE_AES_<xxx>_PKCS_CBC_IV is set.

By using the encryption API ‘SecM_Encryption’ the IV will be used for the AES CBC encryption and will be copied to the output buffer.

Figure 6-2 describes the usage of the explicit initialization vector with the encryption use case. The input buffer consists of the IV (one block size) and the plaintext data.

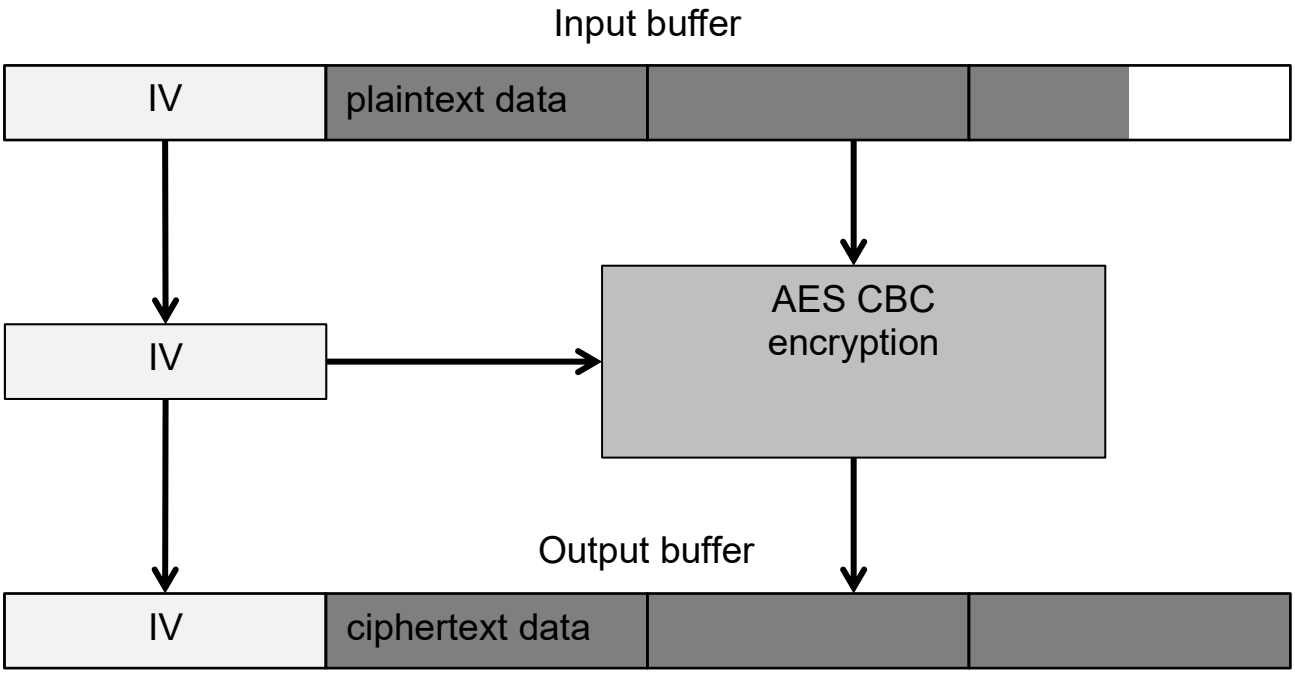


Figure 6-2 Explicit usage of an initialization vector with the encryption API

By the usage of an explicit initialization vector, the first block will be used as the initialization vector and will be copied to the output buffer. Due to the padding mode the output data will be padded to the next multiple of the block size. If the plaintext is aligned to the 16 byte block size an additional padding block will be added.

Using an explicit initialization vector with the decryption API is shown in the Figure 6-3.

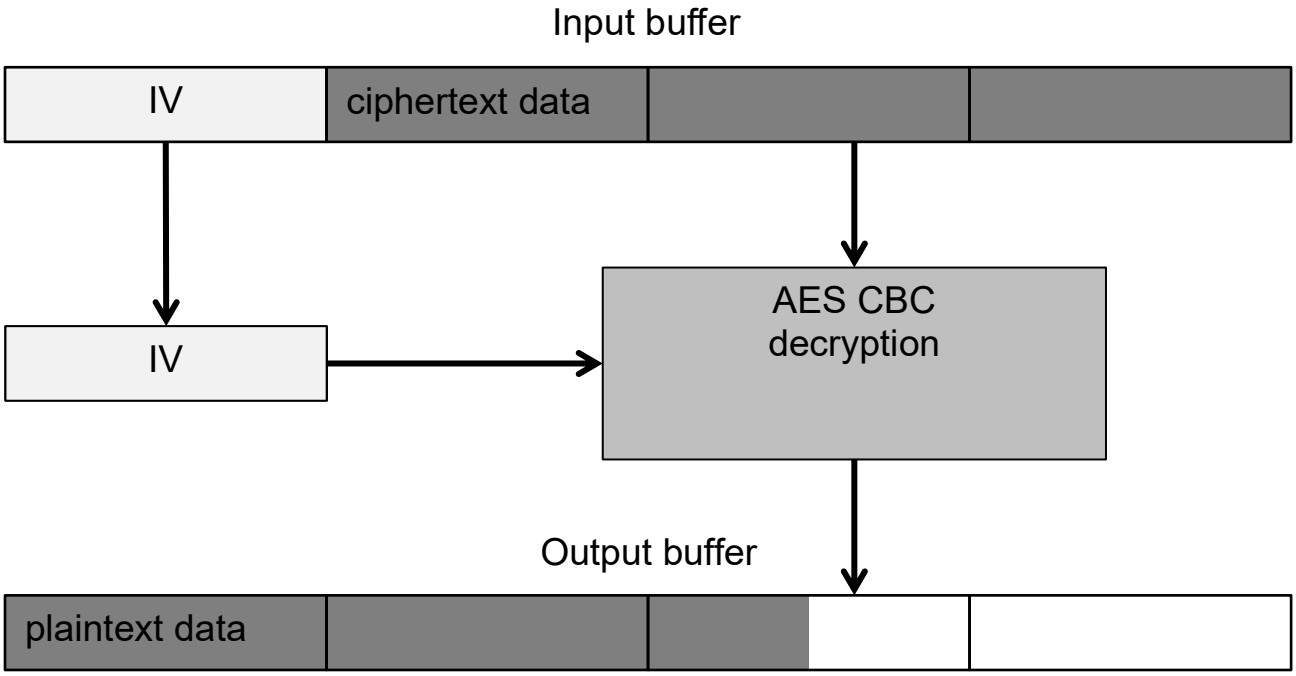


Figure 6-3 Explicit usage of an initialization vector with the decryption API

Similar to the encryption API the decryption mode will treat the first 16 bytes as the IV. This has the effect that no result will be written to the output buffer when the first round provides an input buffer size of less than 32 bytes. Hence the output buffer data will be smaller as the input buffer data due to the removed padding from the result and the obsolete IV data.

6.4.3 Internal / external Key

An external key can be passed to the decryption / encryption. When the provided key.size is equal to zero, the internal key is used for encryption / decryption, otherwise the provided key is used.

6.4.4 Operation mode flags

The operation mode is set by the member ‘mode’ of the function parameter ‘pEncParam’ respectively ‘pDecParam’.

On these operation modes the CBC initialization vector will be filled with the default IV value configured in the cryptographic keys. No additional data have to be passed with the input data buffer:

- SEC_DECRYPTION_MODE_AES_128_PKCS_CBC
- SEC_DECRYPTION_MODE_AES_192_PKCS_CBC
- SEC_DECRYPTION_MODE_AES_256_PKCS_CBC

Using an explicit initialization vector requires one of these flags:

- SEC_DECRYPTION_MODE_AES_128_PKCS_CBC_IV
- SEC_DECRYPTION_MODE_AES_192_PKCS_CBC_IV

> SEC_DECRYPTION_MODE_AES_256_PKCS_CBC_IV

In any case it is required that the cryptographic key references an IV crypto key. If this reference is missing the



Caution
In any case it is required that the cryptographic key references an IV crypto key. If this reference is missing the IV will not be set.

6.4.5 Operation mode modifiers

The operation mode modifier triggers the initialization or the finalization of the AES operation. These modifiers can be combined with the operation mode flags by an OR-operation.

> SEC_DECRYPTION_MODE_INIT

This flag will initialize the workspace, set up the key and copy the IV if necessary.

> SEC_DECRYPTION_MODE_FINALIZE

Setting the flag will process the last input data as well as applying the padding.

6.5 Seed / Key

The Seed/Key interface provides the generation of a pseudo random seed, the generation of a key and a comparison function.

6.5.1 Variants

> Default and extended API

> OEM specific Seed/Key generation

6.5.2 Default and extended API

Depending on the OEM either the default API or the extended API is being used.

6.5.3 Seed generation

The seed can be generated in two different ways:

> Internal: LCG-algorithm (linear congruential generator)

> External: CSM-module

For the LCG-algorithm the multiplier and the increment can be configured.

If the CSM module is selected, the referenced CSM job is used to generate a seed. Keep in mind that in this case an entropy is required.

6.5.4 Key computation

The key is computed externally using the SecM_ComputeKeyVendor (or SecM_CompareKeyVendor in case it is externally computed and compared).

The secret key is being used by the underlying seed/key algorithm.

7 API Description

7.1 Type Definitions

This chapter describes the types used by the interface functions of the security module.

7.1.1 General Types

Typedef	Elements	Basetype	Description
SecM_StatusType	Not particularly defined. Typically, returns SECM_OK or SECM_NOT_OK. See function description for details.	vuint8	Return value from any function of the security module
SecM_WordType	-	vuint32	Typically used by the seed/key functions for seed and key values.
SecM_KeyType	-	SecM_WordType	The length of the key for seed/key calculation.
SecM_CRCType	-	SecM_WordType	Used to store the CRC
SecM_LengthType	-	vuint32	Used to store length information for CRC calculation and decryption
SecM_AddrType	-	vuint32	Used to store address information for segments
SecM_SizeType	-	vuint32	Used to store length information of segments
SecM_VerifyInitType	-	void *	-
SecM_VerifyDeinitType	-	void *	-
SecM_VerifyDataType	-	vuint8 *	-
FL_WDTriggerFctType	typedef void (*)(void);		Pointer to a function that is frequently called from the security module on long-lasting mathematical functions.
FL_ReadMemoryFctType	typedef SecM_SizeType (*)(SecM_AddrType, vuint8 *, SecM_SizeType);		Function is used by SecM_Verification() to read data from memory.

Table 7-1 Type definitions

7.1.2 Structures used by Verification

Structure	Element	Type	Description
SecM_Verify ParamType	segmentList	FL_SegmentListType / FL_SegmentListType* [1]	Address and length information of download segments
	blockStartAddress	SecM_AddrType	Start address of logical block
	blockLength	SecM_SizeType	Length of logical block
	verificationData	SecM_VerifyDataType	Pointer to verification data (e.g. checksum and/or signature)
	crcTotal	SecM_CRCType	CRC total calculated over full logical block
	wdTriggerFct	FL_WDTriggerFctType	Pointer to watchdog trigger function
	workspace	SecM_WorkspaceType	Reference to workspace (extension of the HIS specification)
	key	SecM_VerifyKeyType	Pointer to verification key (extension of the HIS specification)

Table 7-2 SecM_VerifyParamType

Structure	Element	Type	Description
SecM_Signature ParamType	currentHash	SecM_SignatureType	Reference to current hash value (also used to pass workspace)
	currentDataLength	SecM_SizeType *	Pointer to the current length of the hashed data. Is only set when SEC_ENABLE_VERIFICATION_DATA_LENGTH is activated.
	sigState	SecM_StatusType	Signature state / operation to be executed
	sigSourceBuffer	SecM_VerifyDataType	Pointer to input or verification data
	sigByteCount	SecM_LengthType	Size of input or verification data
	wdTriggerFct	FL_WDTriggerFctType	Pointer to watchdog trigger function

[1] As an extension to the HIS specification the segment list can be used as a pointer. This depends on the OEM.

Structure	Element	Type	Description
	key	SecM_VerifyKeyType	Pointer to verification key (extension of the HIS specification)

Table 7-3 SecM_SignatureParamType

7.1.3 Structures used by CRC

Structure	Element	Type	Description
SecM_CRCParam Type	currentCRC	SecM_CRCType	Current CRC-value
	crcState	SecM_ByteType	Operation state to be carried out
	crcSourceBuffer	SecM_ConstRamData Type	Pointer to source data
	crcByteCount	SecM_LengthType	Number of bytes in source buffer
	wdTriggerFct	FL_WDTriggerFctType	Watchdog trigger function

Table 7-4 SecM_CRCParamType

7.1.4 Structures used by Encryption / Decryption

Structure	Element	Type	Description
SecM_DecInput ParamType	DataBuffer	SecM_ConstRamData Type	Pointer to input data
	Length	SecM_LengthType	Length of input data

Table 7-5 SecM_DecInputParamType

Structure	Element	Type	Description
SecM_DecOutput ParamType	DataBuffer	SecM_RamDataType	Pointer to output data
	Length	SecM_LengthType	Length of output buffer / data

Table 7-6 SecM_DecOutputParamType

Structure	Element	Type	Description
SecM_DecParam Type	segmentAddress	SecM_AddrType	Start address of the segment
	segmentLength	SecM_SizeType	Length of segment in bytes
	mode	SecM_ByteType	Data format ID of encryption and compression
	wdTriggerFct	FL_WDTriggerFctType	Pointer to watchdog trigger function

Structure	Element	Type	Description
	workspace	SecM_WorkspaceType	Reference to workspace (extension of the HIS specification)
	key	SecM_SymKeyType	Reference to decryption key (extension of the HIS specification)

Table 7-7 SecM_DecParamType

7.1.5 Structures used by Seed / Key

Structure	Element	Type	Description
SecM_KeyType	data	SecM_KeyBaseType *	Pointer to actual key data
	context	SecM_VoidPtrType	Additional context information, e.g. workspace

Table 7-8 SecM_KeyType

7.2 Common

7.2.1 SecM_InitPowerOn

Prototype	
`SecM_StatusType` **`SecM_InitPowerOn`** `(SecM_InitType initParam)`	
Parameter	
SecM_InitType initParam	Initialization parameters (unused - reserved for future use)
Return Code	
SecM_StatusType	SECM_OK if initialization successful SECM_NOT_OK otherwise
Functional Description	
Initialize security module	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-9 SecM_InitPowerOn

7.2.2 SecM_Task

Prototype	
`void SecM_Task (void)`	
Parameter	
void	none
Return Code	
void	none
Functional Description	
Cyclic task of security module	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-10 SecM_Task

7.3 CRC

7.3.1 SecM_ComputeCRC

Prototype	
`SecM_StatusType SecM_ComputeCRC (V_MEMRAM1 SecM_CRCParamType V_MEMRAM2 V_MEMRAM3 *crcParam)`	
Parameter	
SecM_CRCParamType crcParam	Pointer to parameter structure
Return Code	
SecM_StatusType	SECM_OK if operation was successful SECM_NOT_OK otherwise
Functional Description	
Function that manages the state of CRC computation	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-11 SecM_ComputeCRC

7.4 Encryption / Decryption

7.4.1 SecM_InitDecryption

Prototype	
`SecM_StatusType` **`SecM_InitDecryption`** `(SecM_DecInitType init)`	
Parameter	
SecM_DecInitType init	Dummy pointer (currently not used)
Return Code	
SecM_StatusType	SECM_OK if initialization successful SECM_NOT_OK if error occurred during initialization
Functional Description	
Initialize decryption	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-12 SecM_InitDecryption

7.4.2 SecM_DeinitDecryption

Prototype	
`SecM_StatusType` **`SecM_DeinitDecryption`** `(SecM_DecDeinitType deinit)`	
Parameter	
SecM_DecDeinitType deinit	Dummy pointer (currently not used)
Return Code	
SecM_StatusType	SECM_OK if deinitialization successful SECM_NOT_OK if error occurred during deinitialization
Functional Description	
Deinitialize decryption	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-13 SecM_DeinitDecryption

7.4.3 SecM_Decryption

Prototype	
`SecM_StatusType SecM_Decryption (const V_MEMRAM1 SecM_DecInputParamType V_MEMRAM2 V_MEMRAM3 *pInBlock, V_MEMRAM1 SecM_DecOutputParamType V_MEMRAM2 V_MEMRAM3 *pOutBlock, V_MEMRAM1 SecM_DecParamType V_MEMRAM2 V_MEMRAM3 *pDecParam)`	
Parameter	
SecM_DecInputParamType pInBlock	Reference to input data
SecM_DecOutputParamType pOutBlock	Reference to output data. Length contains available buffer size as input and length of decrypted data as output
SecM_DecParamType pDecParam	Decryption parameters (segmentAddress and segmentLength not used yet)
Return Code	
SecM_StatusType	SECM_OK if decryption successful SECM_NOT_OK if error occurred during decryption
Functional Description	
Decrypts all encrypted data referenced in input block and stores it into the buffer provided in output block. Operation reports an error in case the output buffer isn't large enough.	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-14 SecM_Decryption

7.4.4 SecM_InitEncryption

Prototype	
`SecM_StatusType` **`SecM_InitEncryption`** `(SecM_EncInitType init)`	
Parameter	
SecM_EncInitType init	Dummy pointer (currently not used)
Return Code	
SecM_StatusType	SECM_OK if initialization successful SECM_NOT_OK if error occurred during initialization
Functional Description	
Initialize encryption	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-15 SecM_InitEncryption

7.4.5 SecM_DeinitEncryption

Prototype	
`SecM_StatusType` **`SecM_DeinitEncryption`** `(SecM_EncDeinitType deinit)`	
Parameter	
SecM_EncDeinitType deinit	Dummy pointer (currently not used)
Return Code	
SecM_StatusType	SECM_OK if deinitialization successful SECM_NOT_OK if error occurred during deinitialization
Functional Description	
Deinitialize encryption	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-16 SecM_DeinitEncryption

7.4.6 SecM_Encryption

Prototype	
`SecM_StatusType SecM_Encryption (const V_MEMRAM1 SecM_EncInputParamType V_MEMRAM2 V_MEMRAM3 *pInBlock, V_MEMRAM1 SecM_EncOutputParamType V_MEMRAM2 V_MEMRAM3 *pOutBlock, V_MEMRAM1 SecM_EncParamType V_MEMRAM2 V_MEMRAM3 *pEncParam)`	
Parameter	
SecM_EncInputParamType pInBlock	Reference to input data
SecM_EncOutputParamType pOutBlock	Reference to output data. Length contains available buffer size as input and length of encrypted data as output
SecM_EncParamType pEncParam	Encryption parameters (segmentAddress and segmentLength not used yet)
Return Code	
SecM_StatusType	SECM_OK if decryption successful SECM_NOT_OK if error occurred during decryption
Functional Description	
Encrypts all data referenced in input block and stores it into the buffer provided in output block. Operation reports an error in case the output buffer isn't large enough.	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-17 SecM_Encryption

7.5 Seed / Key Common

7.5.1 SecM_GenerateSeed

Prototype	
`SecM_StatusType` **`SecM_GenerateSeed`** `(V_MEMRAM1 SecM_SeedType V_MEMRAM2 V_MEMRAM3 *seed)`	
Parameter	
SecM_SeedType seed	Pointer to seed, where the random values shall be stored
Return Code	
SecM_StatusType	SECM_OK
Functional Description	
Uses a pseudo random number generator and an initial seed to generate a seed value. In case of HIS specification 1.0.1 or earlier the last seed is stored in an internal structure.	
Particularities and Limitations	
none	
Pre-Conditions	
The values of seedX and seedY (default API) or seed[0] (extended API) should have a random start value.	
Call Context	
none	

Table 7-18 SecM_GenerateSeed

7.5.2 SecM_CompareKey (HIS Specification < 1.0.1[2])

Prototype	
`SecM_StatusType` **`SecM_CompareKey`** `(SecM_KeyType key)`	
Parameter	
SecM_KeyType key	Key for the authorisation
Return Code	
SecM_StatusType	SECM_OK if key calculation and comparison was successful SECM_FAILED if key calculation failed or key mismatch
Functional Description	
Execute the key calculation and compares the calculated key against the received key	
Particularities and Limitations	
none	
Pre-Conditions	
Function SecM_GenerateSeed must have been called before.	
Call Context	
none	

[2] Depending on the OEM

Table 7-19 SecM_CompareKey

7.5.3 SecM_CompareKey (HIS Specification >= 1.0.1)

Prototype	
`SecM_StatusType` **`SecM_CompareKey`** `(SecM_KeyType key, SecM_SeedType lastSeed)`	
Parameter	
SecM_KeyType key	Key for the authorization contains additional parameters (workspace, secret key) in case of extended API
SecM_SeedType lastSeed	Start value (seed) for the authorization
Return Code	
SecM_StatusType	none
Functional Description	
Execute the key calculation and compares the calculated key against the received key	
Particularities and Limitations	
none	
Pre-Conditions	
Function SecM_GenerateSeed must have been called before.	
Call Context	
none	

Table 7-20 SecM_CompareKey

7.6 Seed / Key default API

7.6.1 SecM_ComputeKey

Prototype	
`SecM_StatusType` **`SecM_ComputeKey`** `(SecM_SeedType inputSeed, SecM_ConstType constant, V_MEMRAM1 SecM_KeyStorageType V_MEMRAM2 V_MEMRAM3 *computedKey)`	
Parameter	
SecM_SeedType inputSeed	The random seed the key calculation shall be based on
SecM_ConstType constant	A fixed constant used by the key calculation algorithm contains additional parameters (workspace, secret key) in case of extended API
SecM_KeyStorageType computedKey	Pointer to resulting key data as a formula of key = f(seed,k)
Return Code	
SecM_StatusType	SECM_OK if calculation was successful SECM_FAILED if calculation has failed (e.g. wrong parameters)
Functional Description	
Calculates a key value based on the provided seed and constant value	
Particularities and Limitations	
none	
Pre-Conditions	
Function SecM_GenerateSeed must have been called at least once.	
Call Context	
none	

Table 7-21 SecM_ComputeKey

7.7 Seed / Key extended API

7.7.1 SecM_ComputeKey

Prototype	
`SecM_StatusType` **`SecM_ComputeKey`** `(SecM_KeyType inputKey, SecM_ConstType constant, V_MEMRAM1 SecM_SeedType V_MEMRAM2 V_MEMRAM3 *computedSeed)`	
Parameter	
SecM_KeyType inputKey	The received key on which calculation shall be based
SecM_ConstType constant	A fixed constant used by the key calculation algorithm contains additional parameters (workspace, secret key) in case of extended API
SecM_SeedType computedSeed	Pointer to resulting seed as a formula of seed = f(key,k)
Return Code	
SecM_StatusType	SECM_OK if calculation was successful SECM_FAILED if calculation has failed (e.g. wrong parameters)
Functional Description	
API is a deviation from the HIS specification. Modified to support asymmetric key calculation where only the tester is able to generate the key from the original seed value, e.g. using a private key.	
Particularities and Limitations	
none	
Pre-Conditions	
Function SecM_GenerateSeed must have been called at least once.	
Call Context	
none	

Table 7-22 SecM_ComputeKey

7.8 Verification

7.8.1 SecM_InitVerification

Prototype	
`SecM_StatusType` **`SecM_InitVerification`** `(SecM_VerifyInitType init)`	
Parameter	
SecM_VerifyInitType init	Dummy pointer (currently not used)
Return Code	
SecM_StatusType	SECM_OK if initalization successful SECM_NOT_OK if error occured during initalization
Functional Description	
Initializes the verification	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-23 SecM_InitVerification

7.8.2 SecM_DeinitVerification

Prototype	
`SecM_StatusType` **`SecM_DeinitVerification`** `(SecM_VerifyDeinitType deinit)`	
Parameter	
SecM_VerifyDeinitType deinit	Dummy pointer (currently not used)
Return Code	
SecM_StatusType	SECM_OK if deinitalization successful SECM_NOT_OK if error occured during deinitalization
Functional Description	
Deinitializes the verification	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-24 SecM_DeinitVerification

7.8.3 SecM_Verification

Prototype	
`SecM_StatusType SecM_Verification (V_MEMRAM1 SecM_VerifyParamType V_MEMRAM2 V_MEMRAM3 *pVerifyParam)`	
Parameter	
SecM_VerifyParamType pVerifyParam	Pointer to parameter structure for signature verification, crcTotal, crcWritten and Vendor verification.
Return Code	
SecM_StatusType	SECM_VER_OK if signature verification successful SECM_VER_ERROR if error occurred during verification SECM_VER_CRC if CRC verification failed SECM_VER_SIG if MAC/Signature verification failed
Functional Description	
Calculates the checksum or signature of all segments present in the verification parameter function passed in the verification parameter. Function passed in readMemory is used to access memory. Afterwards the calculated value is used to verify the checksum/signature provided in verificationData. If CRCTotal is configured a CRC over the complete block, defined by blockStartAddress and blockLength, including all readable gaps between the actual segments is calculated too and returned in parameter	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-25 SecM_Verification

7.8.4 SecM_VerifySignature

Prototype	
`SecM_StatusType SecM_VerifySignature (V_MEMRAM1 SecM_SignatureParamType V_MEMRAM2 V_MEMRAM3 *pVerifyParam)`	
Parameter	
pVerifyParam	Pointer to parameter structure for signature verification
Return Code	
SecM_StatusType	SECM_VER_OK if verification operation successful SECM_VER_ERROR if error occurred during verification SECM_VER_CRC if CRC verification failed SECM_VER_SIG if HMAC/Signature verification failed
Functional Description	
Calculate and verify checksum or signature using primitive of configured security class. Perform one of the following operations, defined by pVerifyParam->sigState: > Initialization (kHashInit) > Update checksum (kHashCompute) > Finalize checksum (kHashFinalize) > Verify signature/checksum (kSigVerify)	
Particularities and Limitations	
none	
Pre-Conditions	
Call Context	
none	

Table 7-26 SecM_VerifySignature

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com